

# Managing User Usage and Limits

---

This guide covers how to adjust the number of insights or exports a specific user can run, without affecting other users.

## How limits work

Every plan (`anonymous`, `free_tier`, `small`, `medium`, `large`, `promo_medium`) has fixed monthly caps for downloads and insights. Caps live in code (`src/libs/plan-config.ts`), not in the database.

Every time a user exports an Excel file or runs an analysis, the system inserts a row into the `usage_log` table. The Usage & Limits panel reads the rolling last 30 days of `usage_log` for that user, sums the rows, and compares the sum to the cap.

One insight run counts as 1 unit, regardless of selection size. One export counts as N units, equal to the number of records exported.

So a user's remaining allowance is:

```
cap from PLAN_CONFIGS – sum of their usage_log rows in the last 30 days
```

## What you can change, and where

There are three ways to adjust limits, depending on what you want to do:

- To give one user more insights or exports right now: delete rows in `usage_log` for that user (Option 1 below).
- To move someone to a different cap permanently: change their plan (Option 2).
- To change the cap for everyone on a plan: edit `PLAN_CONFIGS` and redeploy (Option 3).

The rest of the guide walks through Option 1 in detail since it's the most common one.

---

## Option 1 — Adjust a single user via `usage_log`

Use this when a customer asks for more insights this month, or when you want to reset a tester back to zero.

### Step 1 — Open the Supabase Table Editor

1. Go to <https://supabase.com/dashboard>
2. Open the InsideFirms project.
3. In the left sidebar click **Table Editor**.
4. Open the `usage_log` table. Its columns are:

| Column | Meaning |
|--------|---------|
|--------|---------|

---

| Column                  | Meaning                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>id</code>         | Internal row id (auto-generated, ignore)                                                                     |
| <code>user_id</code>    | The user this row belongs to (UUID)                                                                          |
| <code>action</code>     | <code>record_download</code> (export), <code>ai_question</code> (insight), or <code>selection_created</code> |
| <code>count</code>      | How many units this row consumes (1 for an insight, N for an export of N records)                            |
| <code>created_at</code> | When the action happened                                                                                     |

## Step 2 — Find the user's `user_id`

You need the UUID before touching `usage_log`.

1. In the Table Editor, switch the schema dropdown at the top from `public` to `auth`.
2. Open the `auth.users` table.
3. Use the Filter button to filter by `email = the user's address`.
4. Copy the `id` value of that row. That UUID is the `user_id` for the next step.

The user can also see their UUID on their own Settings page under "User ID".

## Step 3 — Decide what to delete

A few common cases:

- Give them their full quota back this month: delete all their rows in `usage_log` from the last 30 days. Their counter goes back to 0.
- Give them a few more insights: delete only the most recent `ai_question` rows until the counter drops to where you want it.
- Give them more exports specifically: delete `record_download` rows; leave `ai_question` rows alone.
- Reset a tester completely: delete every row for that `user_id`. They'll look brand-new in the panel.

## Step 4 — Filter `usage_log` to the right rows

1. Switch the schema dropdown back to `public`.
2. Open `usage_log`.
3. Click Filter and add: `user_id = <UUID from Step 2>`.
4. Optionally narrow further with a second filter:
  - `action = ai_question` for insights only
  - `action = record_download` for exports only
  - `created_at >= now() - interval '30 days'` for rows that actually count toward the current cap (older rows already don't count, so deleting them does nothing)

You should now see only the rows you intend to delete.

## Step 5 — Delete the rows

1. Tick the checkbox at the left of each row, or use the header checkbox to select all visible.
2. Click **Delete rows** (the button appears once any row is selected).
3. Confirm.

The Usage & Limits panel for that user will reflect the new total on their next refresh. There is no cache to invalidate and no job to trigger.

## Step 6 — Verify

Either ask the user to refresh their **Settings → Usage & Limits** page, or in Supabase open the SQL Editor and run:

```
select * from public.get_usage_stats('<UUID> '::uuid);
```

The returned **downloads** and **ai\_calls** are the new totals.

---

## Option 2 — Change someone's plan

If you'd rather give them a higher cap permanently or for the next billing cycle, change their plan instead of editing **usage\_log**.

The cleanest way is the Stripe Customer Portal:

1. Open Stripe Dashboard → Customers.
2. Find the customer by email.
3. Subscriptions tab → upgrade them to a higher plan, or apply a coupon.

For an ad-hoc change with no billing involved (e.g. an internal tester), edit the **subscriptions** table directly:

1. Supabase Table Editor → **subscriptions**.
2. Filter by **user\_id**.
3. Change **price\_id** to the price id that corresponds to the desired plan. You can find Stripe price ids in the **prices** table. Status should stay **active**.

The user picks up the new plan on their next page load. **getUserPlan()** reads from **subscriptions** on every request, so there's no delay.

---

## Option 3 — Change the cap for an entire plan

This is a code change, not something you do per user.

1. Open **src/libs/plan-config.ts**.
2. Edit the relevant entry, for example:

```
medium: {
  maxDownloadsPer30Days: 2000,
  maxAiCallsPer30Days: 1000,
  topKLimit: 500,
},
```

3. Commit, push, redeploy. Every user on **medium** picks up the new cap once the deployment finishes.

Don't use this for single-user adjustments. It changes the cap for everyone on that plan.

---

## A few things worth knowing

**usage\_log** rows older than 30 days are ignored by the limit calculation. Deleting them is fine for tidiness but doesn't change anyone's allowance.

Deleting rows takes effect immediately. The next call to **get\_usage\_stats()** (i.e. on the next page refresh) sees the new total.

Editing **count** in place is technically possible but deletion is cleaner. It leaves an audit-friendly state instead of a mutated row.

The **users.credits** column is not used by the app today. It's reserved for the future Pay-as-you-go model. Setting it to a number does not change any limits.

## Where this maps in the code

- Plan caps: **src/libs/plan-config.ts**
- Usage check on every API: **src/libs/usage-tracking.ts**
- Postgres function:  
**supabase/migrations/20251211164000\_create\_usage\_functions.sql**  
(**get\_usage\_stats**, **check\_usage\_limit**)
- UI counter: **src/components/usage/usage-meter.tsx** (rendered on the Settings page)